Lecture 3. Higher-Order Functions and Lists

Programming Paradigms, Fall 2026 — 9 Sep 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. Where We Are

2. Higher-Order Functions

3. Exercises

Where We Are

This Lecture

Last week: lists, recursion over them, the accumulator parameter.

This Lecture

Last week: lists, recursion over them, the accumulator parameter.

This week: the recursion patterns become functions.

This Lecture

Last week: lists, recursion over them, the accumulator parameter.

This week: the recursion patterns become functions.

Three things to get out of it:

1. **Using** the standard higher-order functions,
   and **reading** code written with them.
2. **Implementing** a higher-order function from its specification.
3. **Factoring out** the shared logic of several functions into one.

Lexical Scope: Racket and Python

A tax rate typed by the user, and a function that uses it:

```
(define rate (read))   ; user types 20
(define (with-tax price)
  (* price (+ 1 (/ rate 100))))
(define (estimate price)
  (let ([rate 0]) (with-tax price)))
(estimate 100)         ; 120
```

```
rate = int(input())    # user types 20
def with_tax(price):
    return price * (1 + rate / 100)
def estimate(price):
    rate = 0
    return with_tax(price)
estimate(100)          # 120.0
```

Lexical Scope: Racket and Python

A tax rate typed by the user, and a function that uses it:

```
(define rate (read))   ; user types 20
(define (with-tax price)
  (* price (+ 1 (/ rate 100))))
(define (estimate price)
  (let ([rate 0]) (with-tax price)))
(estimate 100)         ; 120
```

```
rate = int(input())    # user types 20
def with_tax(price):
    return price * (1 + rate / 100)
def estimate(price):
    rate = 0
    return with_tax(price)
estimate(100)          # 120.0
```

Lexical scope:

- the `rate` in `with-tax` is the one visible where `with-tax` was *written*;
- that is the module's, whatever the user typed;
- the `rate` of `estimate` is invisible to it.

Dynamic Binding: Bash, and JavaScript's `this`

```bash
read rate              # user types 20
with_tax() {
  echo $(( $1 + $1 * rate / 100 )); }
estimate() {
  local rate=0; with_tax $1; }
estimate 100           # 100
```

```javascript
const shop = { rate: 20,
  withTax(price) {
    return price * (1 + this.rate / 100);
  } };
shop.withTax(100);     // 120
const f = shop.withTax;
f(100);                // NaN
```

Dynamic Binding: Bash, and JavaScript's `this`

```bash
read rate              # user types 20
with_tax() {
  echo $(( $1 + $1 * rate / 100 )); }
estimate() {
  local rate=0; with_tax $1; }
estimate 100           # 100
```

```javascript
const shop = { rate: 20,
  withTax(price) {
    return price * (1 + this.rate / 100);
  } };
shop.withTax(100);     // 120
const f = shop.withTax;
f(100);                // NaN
```

Dynamic binding:

- bash looks `rate` up in the *caller*, so the **`local`** of `estimate` wins;
- JavaScript's **`this`** is bound by the *call*, not by where `withTax` was written;
- it is the object before the dot, or nothing.

Why Would an Engineer Reason With the Model?

Every IDE offers *extract variable*, here on a price fetched from a service:

```
# before
if fetch_price(item) > limit:
    alert(item, fetch_price(item))
```

```
# after
p = fetch_price(item)
if p > limit: alert(item, p)
```

Why Would an Engineer Reason With the Model?

Every IDE offers *extract variable*, here on a price fetched from a service:

```
# before
if fetch_price(item) > limit:
    alert(item, fetch_price(item))
```

```
# after
p = fetch_price(item)
if p > limit: alert(item, p)
```

In the model the two are the same.
In production, one request instead of two, and the price may have moved.
The *after* is probably what you wanted, and the model does not choose.
It only tells you that the two are not interchangeable.

Why Would an Engineer Reason With the Model?

Every IDE offers *extract variable*, here on a price fetched from a service:

```
# before
if fetch_price(item) > limit:
    alert(item, fetch_price(item))
```

```
# after
p = fetch_price(item)
if p > limit: alert(item, p)
```

In the model the two are the same.
In production, one request instead of two, and the price may have moved.
The *after* is probably what you wanted, and the model does not choose.
It only tells you that the two are not interchangeable.

The model says when such a replacement is **safe**: no effects inside.

Why Would an Engineer Reason With the Model?

Every IDE offers *extract variable*, here on a price fetched from a service:

```
# before
if fetch_price(item) > limit:
    alert(item, fetch_price(item))
```

```
# after
p = fetch_price(item)
if p > limit: alert(item, p)
```

In the model the two are the same.
In production, one request instead of two, and the price may have moved.
The *after* is probably what you wanted, and the model does not choose.
It only tells you that the two are not interchangeable.

The model says when such a replacement is **safe**: no effects inside.

1. **Refactoring.** Extracting a variable is safe when the model says so.

Why Would an Engineer Reason With the Model?

Every IDE offers *extract variable*, here on a price fetched from a service:

```
# before
if fetch_price(item) > limit:
    alert(item, fetch_price(item))
```

```
# after
p = fetch_price(item)
if p > limit: alert(item, p)
```

In the model the two are the same.
In production, one request instead of two, and the price may have moved.
The *after* is probably what you wanted, and the model does not choose.
It only tells you that the two are not interchangeable.

The model says when such a replacement is **safe**: no effects inside.

1. **Refactoring.** Extracting a variable is safe when the model says so.
2. **Code review.** A rewrite is right for *every* input, not for three tests.

Why Would an Engineer Reason With the Model?

Every IDE offers *extract variable*, here on a price fetched from a service:

```
# before
if fetch_price(item) > limit:
    alert(item, fetch_price(item))
```

```
# after
p = fetch_price(item)
if p > limit: alert(item, p)
```

In the model the two are the same.
In production, one request instead of two, and the price may have moved.
The *after* is probably what you wanted, and the model does not choose.
It only tells you that the two are not interchangeable.

The model says when such a replacement is **safe**: no effects inside.

1. **Refactoring.** Extracting a variable is safe when the model says so.
2. **Code review.** A rewrite is right for *every* input, not for three tests.
3. **Testing.** A claim for every list is a *property*, and properties can be tested.

Higher-Order Functions

About Higher-Order Functions

A **higher-order function** (in programming) is any function that accepts another function as an argument or returns a function as its return value.

```
(define (twice f x)
  (f (f x)))

(twice sqrt 16)  ; 2
```

About Higher-Order Functions

A **higher-order function** (in programming) is any function that accepts another function as an argument or returns a function as its return value.

```
(define (twice f x)
  (f (f x)))

(twice sqrt 16)  ; 2
```

We will care about higher-order functions from different perspectives:

1. Using several *standard* higher-order functions
2. Implementing higher-order functions given their specification
3. Reducing boilerplate in user code by factoring out (higher-order) logic

Higher-Order Functions: `map`

Function `map` applies a given function to every element in a given list, producing a (new) list of results:

```
(map (lambda (x) (* x x))
     '(1 2 3 4 5))
; '(1 4 9 16 25)

(map sqr '(1 2 3 4 5)) ; the same: sqr is built in
```

Higher-Order Functions: `map`



Function `map` applies a given function to every element in a given list, producing a (new) list of results:

```
(map (lambda (x) (* x x))
     '(1 2 3 4 5))
; '(1 4 9 16 25)

(map sqr '(1 2 3 4 5)) ; the same: sqr is built in
```

This function is often used as part of a larger processing: first we process each element in a collection, then we often combine the results somehow.

Higher-Order Functions: `map`

Function `map` applies a given function to every element in a given list, producing a (new) list of results:

```
(map (lambda (x) (* x x))
     '(1 2 3 4 5))
; '(1 4 9 16 25)

(map sqr '(1 2 3 4 5)) ; the same: sqr is built in
```

This function is often used as part of a larger processing: first we process each element in a collection, then we often combine the results somehow.

In declarative programming, it is useful to *think* of `map` as an *atomic* operation, as if it processes an entire list in one go.

First, the Logical Operators

Logical operators[1] **`or`** and **`and`** are

- *polyvariadic*: they take any number of arguments.
- *short-circuiting* ("lazy"): evaluation stops once the result is known.
- *special forms* (macros), **not** functions: they have to be fully applied.

```
(define (fib n)
  (cond
    [(<= n 2) 1]
    [else (+ (fib (- n 1)) (fib (- n 2)))]))
(or  (> 3 2) (> (fib 100) 1000))   ; #t
(and (> 2 3) (> (fib 100) 1000))   ; #f
```

[1] See Racket Essentials §2.2.5.

First, the Logical Operators

Logical operators[1] **`or`** and **`and`** are

- *polyvariadic*: they take any number of arguments.
- *short-circuiting* ("lazy"): evaluation stops once the result is known.
- *special forms* (macros), **not** functions: they have to be fully applied.

```
(define (fib n)
  (cond
    [(<= n 2) 1]
    [else (+ (fib (- n 1)) (fib (- n 2)))]))
(or  (> 3 2) (> (fib 100) 1000))   ; #t
(and (> 2 3) (> (fib 100) 1000))   ; #f
```

Square brackets are parentheses: Racket treats `[` and `(` alike, and by convention each **`cond`** clause and each **`let`** binding gets them.

[1] See Racket Essentials §2.2.5.

Higher-Order Functions: `ormap` and `andmap`

Functions `ormap` and `andmap` map a predicate before applying **`or`** or **`and`**.

`ormap` checks if *any* of the elements satisfies the predicate:

```
(ormap odd? '(2 4 6 8 10))
; #f
```

`andmap` checks if *all* elements satisfy the predicate:

```
(andmap even? '(2 4 6 8 10))
; #t
```

Higher-Order Functions: `ormap` and `andmap` (Example)



A cart is a list of items, each a name and a price:

```
(define cart '(("pen" 120) ("notebook" 350) ("bag" 2900)))

(define (any-over? limit cart)
  (ormap (lambda (item) (> (second item) limit)) cart))
(define (all-under? limit cart)
  (andmap (lambda (item) (<= (second item) limit)) cart))
(any-over? 1000 cart)  ; #t
(all-under? 3000 cart) ; #t
```

Higher-Order Functions: `ormap` and `andmap` (Example)



A cart is a list of items, each a name and a price:

```
(define cart '(("pen" 120) ("notebook" 350) ("bag" 2900)))

(define (any-over? limit cart)
  (ormap (lambda (item) (> (second item) limit)) cart))
(define (all-under? limit cart)
  (andmap (lambda (item) (<= (second item) limit)) cart))
(any-over? 1000 cart)  ; #t
(all-under? 3000 cart) ; #t
```

Where does `limit` come from?

The `lambda`'s only parameter is `item`.
Its `limit` is the enclosing function's.

Higher-Order Functions: `apply`

`apply` calls a function with the elements of a list as its arguments:

```
(apply + '(1 2 3 4))                ; (+ 1 2 3 4)          = 10
(apply max '(3 9 4))                ; (max 3 9 4)          = 9
(apply string-append '("a" "b"))    ; (string-append "a" "b")
```

Higher-Order Functions: `apply`

`apply` calls a function with the elements of a list as its arguments:

```
(apply + '(1 2 3 4))                ; (+ 1 2 3 4)          = 10
(apply max '(3 9 4))                ; (max 3 9 4)          = 9
(apply string-append '("a" "b"))    ; (string-append "a" "b")
```

Useful when the arguments come as a list.
It works for any function that takes any number of arguments.

Higher-Order Functions: `apply`

`apply` calls a function with the elements of a list as its arguments:

```
(apply + '(1 2 3 4))                ; (+ 1 2 3 4)          = 10
(apply max '(3 9 4))                ; (max 3 9 4)          = 9
(apply string-append '("a" "b"))    ; (string-append "a" "b")
```

Useful when the arguments come as a list.
It works for any function that takes any number of arguments.

Question

`or` takes any number of arguments. Does `(apply or lst)` work?

Higher-Order Functions: Implementing `ormap`



The following definition does not work. Why?

```
(define (my-ormap f lst)
  (apply or (map f lst)))
```

Higher-Order Functions: Implementing `ormap`

The following definition does not work. Why?

```
(define (my-ormap f lst)
  (apply or (map f lst)))
```

`or` is a special form, not a function.
It cannot be passed as an argument, so `(apply or ...)` is a syntax error.

Higher-Order Functions: Implementing `ormap`

The following definition does not work. Why?

```
(define (my-ormap f lst)
  (apply or (map f lst)))
```

`or` is a special form, not a function.
It cannot be passed as an argument, so `(apply or ...)` is a syntax error.

Is it possible to implement `my-ormap` in some other way?

Higher-Order Functions: Implementing `ormap`

The following definition does not work. Why?

```
(define (my-ormap f lst)
  (apply or (map f lst)))
```

`or` is a special form, not a function.
It cannot be passed as an argument, so `(apply or ...)` is a syntax error.

Is it possible to implement `my-ormap` in some other way?
By explicit recursion, with **`or`** applied in the body:

```
(define (my-ormap f lst)
  (cond [(empty? lst) #f]
        [else (or (f (first lst)) (my-ormap f (rest lst)))]))
```

Higher-Order Functions: Implementing `ormap`

The following definition does not work. Why?

```
(define (my-ormap f lst)
  (apply or (map f lst)))
```

`or` is a special form, not a function.
It cannot be passed as an argument, so `(apply or ...)` is a syntax error.

Is it possible to implement `my-ormap` in some other way?
By explicit recursion, with **`or`** applied in the body:

```
(define (my-ormap f lst)
  (cond [(empty? lst) #f]
        [else (or (f (first lst)) (my-ormap f (rest lst)))]))
```

Does it still stop at the first element that passes?

Higher-Order Functions: Implementing `ormap`

The following definition does not work. Why?

```
(define (my-ormap f lst)
  (apply or (map f lst)))
```

`or` is a special form, not a function.
It cannot be passed as an argument, so `(apply or ...)` is a syntax error.

Is it possible to implement `my-ormap` in some other way?
By explicit recursion, with **`or`** applied in the body:

```
(define (my-ormap f lst)
  (cond [(empty? lst) #f]
        [else (or (f (first lst)) (my-ormap f (rest lst)))]))
```

Does it still stop at the first element that passes?
Yes. **`or`** evaluates the recursive call only if `(f (first lst))` is false.

Higher-Order Functions: `filter`

Function `filter` produces a sublist, containing only elements of the original list that satisfy the predicate:

```
(filter even? '(1 2 3 4 5))
; '(2 4)

(filter prime? (range 2 50))
; '(2 3 5 7 11 13 17 19 23 29 31 37 41 43 47)
```

Higher-Order Functions: `filter`



Function `filter` produces a sublist, containing only elements of the original list that satisfy the predicate:

```
(filter even? '(1 2 3 4 5))
; '(2 4)

(filter prime? (range 2 50))
; '(2 3 5 7 11 13 17 19 23 29 31 37 41 43 47)
```

Again, similarly to function `map`, it is useful to *think* of `filter` as an *atomic* operation, as if it processes an entire list in one go.

Higher-Order Functions: `map` and `filter` (Example 1 of 3)

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))
```

```
students
; '(("Anna" . 4.3) ("Boris" . 3.6)
;   ("Charles" . 4.1) ("Daria" . 2.9))
```

Higher-Order Functions: `map` and `filter` (Example 2 of 3)

```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))


    (filter (lambda (student)
              (> (cdr student) 4.0))
            students)
; '(("Anna" . 4.3) ("Charles" . 4.1))
```

Higher-Order Functions: `map` and `filter` (Example 3 of 3)



```
(define students
  (list
   (cons "Anna"    4.3)
   (cons "Boris"   3.6)
   (cons "Charles" 4.1)
   (cons "Daria"   2.9)))

(map car
     (filter (lambda (student)
               (> (cdr student) 4.0))
             students))
; '("Anna" "Charles")
```

Reading a Pipeline Out Loud

```
(map car
     (filter (lambda (student) (> (cdr student) 4.0))
             students))
```

Read it **inside out**, from the data to the result:

Reading a Pipeline Out Loud

```
(map car
     (filter (lambda (student) (> (cdr student) 4.0))
             students))
```

Read it **inside out**, from the data to the result:

1. `students`: the list of pairs.

Reading a Pipeline Out Loud

```
(map car
     (filter (lambda (student) (> (cdr student) 4.0))
             students))
```

Read it **inside out**, from the data to the result:

1. `students`: the list of pairs.
2. `(filter ... students)`: the pairs whose grade is above `4.0`.

Reading a Pipeline Out Loud

```
(map car
     (filter (lambda (student) (> (cdr student) 4.0))
             students))
```

Read it **inside out**, from the data to the result:

1. `students`: the list of pairs.
2. `(filter ... students)`: the pairs whose grade is above `4.0`.
3. `(map car ...)`: from each pair that is left, the name.

Reading a Pipeline Out Loud

```
(map car
     (filter (lambda (student) (> (cdr student) 4.0))
             students))
```

Read it **inside out**, from the data to the result:

1. `students`: the list of pairs.
2. `(filter ... students)`: the pairs whose grade is above `4.0`.
3. `(map car ...)`: from each pair that is left, the name.

Inside out is also the order of evaluation: call by value.

Reading a Pipeline Out Loud

```
(map car
     (filter (lambda (student) (> (cdr student) 4.0))
             students))
```

Read it **inside out**, from the data to the result:

1. `students`: the list of pairs.
2. `(filter ... students)`: the pairs whose grade is above `4.0`.
3. `(map car ...)`: from each pair that is left, the name.

Inside out is also the order of evaluation: call by value.

Question

Read `(filter even? (map sqr '(1 2 3 4)))` out loud, then evaluate it.

Read It Like SQL, Not Like a Loop

The same query, in two languages:

```
(map car
     (filter (lambda (s) (> (cdr s) 4.0))
             students))
```

```
SELECT name
FROM students
WHERE grade > 4.0
```

Read It Like SQL, Not Like a Loop

The same query, in two languages:

```
(map car
     (filter (lambda (s) (> (cdr s) 4.0))
             students))
```

```
SELECT name
FROM students
WHERE grade > 4.0
```

Nobody reads a `SELECT` as a loop over rows.
It says *what* rows and *what* of each row.
The database decides *how*.

Read It Like SQL, Not Like a Loop

The same query, in two languages:

```
(map car
     (filter (lambda (s) (> (cdr s) 4.0))
             students))
```

```
SELECT name
FROM students
WHERE grade > 4.0
```

Nobody reads a `SELECT` as a loop over rows.
It says *what* rows and *what* of each row.
The database decides *how*.

Read `map` and `filter` the same way:

- `filter` is the `WHERE`: which elements stay.
- `map` is the `SELECT`: what is kept of each.
- `foldl`, coming next, is the aggregate: `SUM`, `COUNT`, `MAX`.
- `ormap` and `andmap` are `EXISTS` and `ALL`.

Higher-Order Functions: `foldl`

Function `foldl` collapses a list into a single value: it starts from `z` and updates the value with the stepping function `f` for every element.

```
(define (f x current) ...)
(foldl f z '(a b c d))   ; (f 'd (f 'c (f 'b (f 'a z))))
(foldl + 0 '(1 2 3 4))   ; 10
(foldl * 1 '(1 2 3 4))   ; 24
```

Higher-Order Functions: `foldl`

Function `foldl` collapses a list into a single value: it starts from `z` and updates the value with the stepping function `f` for every element.

```
(define (f x current) ...)
(foldl f z '(a b c d))   ; (f 'd (f 'c (f 'b (f 'a z))))
(foldl + 0 '(1 2 3 4))   ; 10
(foldl * 1 '(1 2 3 4))   ; 24
```

Mind the argument order of `f`: the element first, the **accumulator** second.

Higher-Order Functions: `foldl`

Function `foldl` collapses a list into a single value: it starts from `z` and updates the value with the stepping function `f` for every element.

```
(define (f x current) ...)
(foldl f z '(a b c d))   ; (f 'd (f 'c (f 'b (f 'a z))))
(foldl + 0 '(1 2 3 4))   ; 10
(foldl * 1 '(1 2 3 4))   ; 24
```

Mind the argument order of `f`: the element first, the **accumulator** second.

Two equations say everything about `foldl`:

```
(foldl f z empty)        ; = z
(foldl f z (cons x xs))  ; = (foldl f (f x z) xs)
```

Higher-Order Functions: `foldl` (Example 1 of 4)

```
(foldl
 (lambda (student current)
   (cond
     [(> (cdr student) 4.0)
      (cons (car student) current)]
     [else current]))
 empty
 students)
; iteration #1
;   student = '("Anna" . 4.3)
;   current = '()
```

Higher-Order Functions: `foldl` (Example 2 of 4)

```
(foldl
 (lambda (student current)
   (cond
     [(> (cdr student) 4.0)
      (cons (car student) current)]
     [else current]))
 empty
 students)
; iteration #2
;   student = '("Boris" . 3.6)
;   current = '("Anna")
```

Higher-Order Functions: `foldl` (Example 3 of 4)

```
(foldl
 (lambda (student current)
   (cond
     [(> (cdr student) 4.0)
      (cons (car student) current)]
     [else current]))
 empty
 students)
; iteration #3
;   student = '("Charles" . 4.1)
;   current = '("Anna")
```

Higher-Order Functions: `foldl` (Example 4 of 4)



```
(foldl
 (lambda (student current)
   (cond
     [(> (cdr student) 4.0)
      (cons (car student) current)]
     [else current]))
 empty
 students)
; iteration #4
;   student = '("Daria" . 2.9)
;   current = '("Charles" "Anna")
```

Higher-Order Functions: `foldl` (Discussion 1 of 2)

```
(foldl
 (lambda (student current)
   (cond
     [(> (cdr student) 4.0)
      (cons (car student) current)]
     [else current]))
 empty
 students)
; '("Charles" "Anna")
```

Function `foldl` essentially captures the **accumulator parameter pattern** (with exactly one accumulator) and can be thought of as a purely functional way to implement an imperative `foreach`-loop with local state.

Factoring Out: The Same Helper, Twice

Two helpers from lecture 2, side by side:

```
(define (helper l acc)
  (cond [(empty? l) acc]
        [else (helper (rest l)
                      (+ acc (first l)))]))
(define (my-sum l) (helper l 0))
```

```
(define (helper l acc)
  (cond [(empty? l) acc]
        [else (helper (rest l)
                      (cons (first l) acc))]))
(define (my-reverse l) (helper l '()))
```

Factoring Out: The Same Helper, Twice

Two helpers from lecture 2, side by side:

```
(define (helper l acc)
  (cond [(empty? l) acc]
        [else (helper (rest l)
                      (+ acc (first l)))]))
(define (my-sum l) (helper l 0))
```

```
(define (helper l acc)
  (cond [(empty? l) acc]
        [else (helper (rest l)
                      (cons (first l) acc))]))
(define (my-reverse l) (helper l '()))
```

They differ in two places only: the **step**, and the **start**.

Factoring Out: The Same Helper, Twice

Two helpers from lecture 2, side by side:

```
(define (helper l acc)
  (cond [(empty? l) acc]
        [else (helper (rest l)
                      (+ acc (first l)))]))
(define (my-sum l) (helper l 0))
```

```
(define (helper l acc)
  (cond [(empty? l) acc]
        [else (helper (rest l)
                      (cons (first l) acc))]))
(define (my-reverse l) (helper l '()))
```

They differ in two places only: the **step**, and the **start**.

Make the step a parameter and the start an argument, and one helper serves both:

```
(define (my-sum l)     (foldl (lambda (x acc) (+ acc x)) 0 l))
(define (my-reverse l) (foldl cons '() l))
```

Factoring Out: The Same Helper, Twice

Two helpers from lecture 2, side by side:

```
(define (helper l acc)
  (cond [(empty? l) acc]
        [else (helper (rest l)
                      (+ acc (first l)))]))
(define (my-sum l) (helper l 0))
```

```
(define (helper l acc)
  (cond [(empty? l) acc]
        [else (helper (rest l)
                      (cons (first l) acc))]))
(define (my-reverse l) (helper l '()))
```

They differ in two places only: the **step**, and the **start**.

Make the step a parameter and the start an argument, and one helper serves both:

```
(define (my-sum l)     (foldl (lambda (x acc) (+ acc x)) 0 l))
(define (my-reverse l) (foldl cons '() l))
```

That helper with the step passed in is `foldl`: the shared logic written once.

Pending Work, Side by Side

The naïve `my-sum`[a]

```
(my-sum (list 1 2 3))
```

With `foldl`

```
(foldl + 0 (list 1 2 3))
```

[a] Lecture 2's `my-sum/acc`: the slash is part of the name.

Pending Work, Side by Side

The naïve `my-sum`[a]

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
```

With `foldl`

```
(foldl + 0 (list 1 2 3))
= (foldl + 1 (list 2 3))
```

[a] Lecture 2's `my-sum/acc`: the slash is part of the name.

Pending Work, Side by Side

The naïve `my-sum`[a]

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))
```

With `foldl`

```
(foldl + 0 (list 1 2 3))
= (foldl + 1 (list 2 3))
= (foldl + 3 (list 3))
```

[a] Lecture 2's `my-sum/acc`: the slash is part of the name.

Pending Work, Side by Side

The naïve `my-sum`[a]

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))

= (+ 1 (+ 2 (+ 3 (my-sum empty))))
```

With `foldl`

```
(foldl + 0 (list 1 2 3))
= (foldl + 1 (list 2 3))
= (foldl + 3 (list 3))
= (foldl + 6 empty)
```

[a] Lecture 2's `my-sum/acc`: the slash is part of the name.

Pending Work, Side by Side

The naïve `my-sum`[a]

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))

= (+ 1 (+ 2 (+ 3 (my-sum empty))))
= (+ 1 (+ 2 (+ 3 0)))
```

With `foldl`

```
(foldl + 0 (list 1 2 3))
= (foldl + 1 (list 2 3))
= (foldl + 3 (list 3))
= (foldl + 6 empty)
= 6
```

[a] Lecture 2's `my-sum/acc`: the slash is part of the name.

Pending Work, Side by Side

The naïve `my-sum`[a]

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))

= (+ 1 (+ 2 (+ 3 (my-sum empty))))
= (+ 1 (+ 2 (+ 3 0)))
= 6
```

With `foldl`

```
(foldl + 0 (list 1 2 3))
= (foldl + 1 (list 2 3))
= (foldl + 3 (list 3))
= (foldl + 6 empty)
= 6
```

[a] Lecture 2's `my-sum/acc`: the slash is part of the name.

Pending Work, Side by Side

The naïve `my-sum`[a]

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))

= (+ 1 (+ 2 (+ 3 (my-sum empty))))
= (+ 1 (+ 2 (+ 3 0)))
= 6
```

With `foldl`

```
(foldl + 0 (list 1 2 3))
= (foldl + 1 (list 2 3))
= (foldl + 3 (list 3))
= (foldl + 6 empty)
= 6
```

[a] Lecture 2's `my-sum/acc`: the slash is part of the name.

Right: a number at every step.

Left: every `+` waits for the call inside it, and the expression grows.

Pending Work, Side by Side

The naïve `my-sum`[a]

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))

= (+ 1 (+ 2 (+ 3 (my-sum empty))))
= (+ 1 (+ 2 (+ 3 0)))
= 6
```

With `foldl`

```
(foldl + 0 (list 1 2 3))
= (foldl + 1 (list 2 3))
= (foldl + 3 (list 3))
= (foldl + 6 empty)
= 6
```

[a] Lecture 2's `my-sum/acc`: the slash is part of the name.

Right: a number at every step.
Left: every `+` waits for the call inside it, and the expression grows.

A **tail call** is a call nothing waits for.

Higher-Order Functions: `foldl` (Discussion 2 of 2)

Note that `foldl` produces the entire output, so we have to wait until everything is processed before accessing the first entries in the result.

Higher-Order Functions: `foldl` (Discussion 2 of 2)

Note that `foldl` produces the entire output, so we have to wait until everything is processed before accessing the first entries in the result.

Question

> Can we get results *incrementally*, as needed?

Higher-Order Functions: `foldl` (Discussion 2 of 2)

Note that `foldl` produces the entire output, so we have to wait until everything is processed before accessing the first entries in the result.

Question

> Can we get results *incrementally*, as needed?

That is next week: iterators, streams, and generators.

The Higher-Order Functions So Far

	in words	read declaratively	the loop it replaces
`(map f xs)`	`f` of each element	`SELECT f(x)`	one building a new list
`(filter p? xs)`	the elements that pass `p?`	`WHERE p?(x)`	one with an **`if`** inside
`(foldl f z xs)`	`f` folded into `z`, left to right	`SUM`, `COUNT`, ...	one with a single variable
`(ormap p? xs)`	some element passes `p?`	`EXISTS`	one with an early `return`
`(andmap p? xs)`	every element passes `p?`	`ALL`	the same, on the first failure
`(apply f xs)`	`f` called on the elements	—	not a loop: one call

The Higher-Order Functions So Far

	in words	read declaratively	the loop it replaces
`(map f xs)`	`f` of each element	`SELECT f(x)`	one building a new list
`(filter p? xs)`	the elements that pass `p?`	`WHERE p?(x)`	one with an **`if`** inside
`(foldl f z xs)`	`f` folded into `z`, left to right	`SUM`, `COUNT`, ...	one with a single variable
`(ormap p? xs)`	some element passes `p?`	`EXISTS`	one with an early `return`
`(andmap p? xs)`	every element passes `p?`	`ALL`	the same, on the first failure
`(apply f xs)`	`f` called on the elements	—	not a loop: one call

Each takes the *varying* part as a function and keeps the walk to itself.
Think of each as one operation on the whole list.
Read a nesting of them inside out.

5 min break



Exercises

Exercise: `flatten`

Exercise 3.1

Implement `flatten` using higher-order functions.

```
(flatten '(a (b (c d) e)))
; '(a b c d e)
```

Exercise: `flatten`

Exercise 3.1

Implement `flatten` using higher-order functions.

```
(flatten '(a (b (c d) e)))
; '(a b c d e)
```

Hint

An element is a list or it is not, and `list?` tells which.
A list of lists is joined with `append`, which takes any number of arguments.

Exercise: `flatten` (Solution)

```
(define (my-flatten x)
  (cond [(list? x) (apply append (map my-flatten x))]
        [else (list x)]))
```

Exercise: `flatten` (Solution)

```
(define (my-flatten x)
  (cond [(list? x) (apply append (map my-flatten x))]
        [else (list x)]))
```

```
(my-flatten '(a (b c)))
```

Exercise: `flatten` (Solution)

```
(define (my-flatten x)
  (cond [(list? x) (apply append (map my-flatten x))]
        [else (list x)]))
```

```
(my-flatten '(a (b c)))
= (apply append (map my-flatten '(a (b c))))
```

Exercise: `flatten` (Solution)

```
(define (my-flatten x)
  (cond [(list? x) (apply append (map my-flatten x))]
        [else (list x)]))
```

```
(my-flatten '(a (b c)))
= (apply append (map my-flatten '(a (b c))))
= (apply append (list (my-flatten 'a) (my-flatten '(b c))))
```

Exercise: `flatten` (Solution)

```
(define (my-flatten x)
  (cond [(list? x) (apply append (map my-flatten x))]
        [else (list x)]))
```

```
(my-flatten '(a (b c)))
= (apply append (map my-flatten '(a (b c))))
= (apply append (list (my-flatten 'a) (my-flatten '(b c))))
= (apply append (list '(a) (apply append (list '(b) '(c)))))
```

Exercise: `flatten` (Solution)

```
(define (my-flatten x)
  (cond [(list? x) (apply append (map my-flatten x))]
        [else (list x)]))
```

```
(my-flatten '(a (b c)))
= (apply append (map my-flatten '(a (b c))))
= (apply append (list (my-flatten 'a) (my-flatten '(b c))))
= (apply append (list '(a) (apply append (list '(b) '(c)))))
= (apply append (list '(a) '(b c)))
```

Exercise: `flatten` (Solution)

```
(define (my-flatten x)
  (cond [(list? x) (apply append (map my-flatten x))]
        [else (list x)]))
```

```
(my-flatten '(a (b c)))
= (apply append (map my-flatten '(a (b c))))
= (apply append (list (my-flatten 'a) (my-flatten '(b c))))
= (apply append (list '(a) (apply append (list '(b) '(c)))))
= (apply append (list '(a) '(b c)))
= (append '(a) '(b c)) = '(a b c)
```

Exercise: `flatten` (Solution)

```
(define (my-flatten x)
  (cond [(list? x) (apply append (map my-flatten x))]
        [else (list x)]))
```

```
(my-flatten '(a (b c)))
= (apply append (map my-flatten '(a (b c))))
= (apply append (list (my-flatten 'a) (my-flatten '(b c))))
= (apply append (list '(a) (apply append (list '(b) '(c)))))
= (apply append (list '(a) '(b c)))
= (append '(a) '(b c)) = '(a b c)
```

The recursion is on the *nesting*, not on the list.
`map` walks the list, `my-flatten` goes down a level, and `apply append` joins what comes back.

Exercise: expand

Exercise 3.2

Implement `expand` using higher-order functions.

```
(expand '((a 4) (b 1) (c 2) (a 2) (d 1) (e 4)))
; '(a a a a b c c a a d e e e e)
```

Exercise: expand

Exercise 3.2

Implement `expand` using higher-order functions.

```
(expand '((a 4) (b 1) (c 2) (a 2) (d 1) (e 4)))
; '(a a a a b c c a a d e e e e)
```

Hint

`(make-list 3 'a)` is `'(a a a)`.

Exercise: Insertion Sort

Exercise 3.3

Implement a purely functional version of insertion sort, using higher-order functions.

```
(insertion-sort '(8 4 7 2 6 5 1 3 2))
; '(1 2 3 4 5 6 7 8)
```

Exercise: Insertion Sort

Exercise 3.3

Implement a purely functional version of insertion sort, using higher-order functions.

```
(insertion-sort '(8 4 7 2 6 5 1 3 2))
; '(1 2 3 4 5 6 7 8)
```

Hint

Insertion sort takes the elements one at a time and inserts each into a sorted list so far.
A sorted list so far is an accumulator.

Exercise 3.4

Implement `map` and `foldl` using explicit recursion.

Exercise 3.5 ⋆

Implement `map` using `foldl` and without explicit recursion.

Exercise 3.4

Implement `map` and `foldl` using explicit recursion.

Exercise 3.5 ⋆

Implement `map` using `foldl` and without explicit recursion.

Hint

For `foldl`, its two equations are the two **`cond`** clauses.
For `map` via `foldl`, mind the order of the result.

Implementing `map` by Explicit Recursion (Solution)

```
(define (my-map f lst)
  (cond
    [(empty? lst) empty]
    [else (cons (f (first lst))
                (my-map f (rest lst)))]))
```

Implementing `map` by Explicit Recursion (Solution)

```
(define (my-map f lst)
  (cond
    [(empty? lst) empty]
    [else (cons (f (first lst))
                (my-map f (rest lst)))]))
```

```
(my-map sqr (list 1 2 3))
```

Implementing `map` by Explicit Recursion (Solution)

```
(define (my-map f lst)
  (cond
    [(empty? lst) empty]
    [else (cons (f (first lst))
                (my-map f (rest lst)))]))
```

```
(my-map sqr (list 1 2 3))
= (cons (sqr 1) (my-map sqr (list 2 3)))
= (cons 1 (my-map sqr (list 2 3)))
```

Implementing `map` by Explicit Recursion (Solution)

```
(define (my-map f lst)
  (cond
    [(empty? lst) empty]
    [else (cons (f (first lst))
                (my-map f (rest lst)))]))
```

```
(my-map sqr (list 1 2 3))
= (cons (sqr 1) (my-map sqr (list 2 3)))
= (cons 1 (my-map sqr (list 2 3)))
= (cons 1 (cons 4 (my-map sqr (list 3))))
```

Implementing `map` by Explicit Recursion (Solution)

```
(define (my-map f lst)
  (cond
    [(empty? lst) empty]
    [else (cons (f (first lst))
                (my-map f (rest lst)))]))
```

```
(my-map sqr (list 1 2 3))
= (cons (sqr 1) (my-map sqr (list 2 3)))
= (cons 1 (my-map sqr (list 2 3)))
= (cons 1 (cons 4 (my-map sqr (list 3))))
= (cons 1 (cons 4 (cons 9 (my-map sqr empty))))
```

Implementing `map` by Explicit Recursion (Solution)

```
(define (my-map f lst)
  (cond
    [(empty? lst) empty]
    [else (cons (f (first lst))
                (my-map f (rest lst)))]))
```

```
(my-map sqr (list 1 2 3))
= (cons (sqr 1) (my-map sqr (list 2 3)))
= (cons 1 (my-map sqr (list 2 3)))
= (cons 1 (cons 4 (my-map sqr (list 3))))
= (cons 1 (cons 4 (cons 9 (my-map sqr empty))))
= (cons 1 (cons 4 (cons 9 empty))) = '(1 4 9)
```

Implementing `map` by Explicit Recursion (Solution)

```
(define (my-map f lst)
  (cond
    [(empty? lst) empty]
    [else (cons (f (first lst))
                (my-map f (rest lst)))]))
```

```
(my-map sqr (list 1 2 3))
= (cons (sqr 1) (my-map sqr (list 2 3)))
= (cons 1 (my-map sqr (list 2 3)))
= (cons 1 (cons 4 (my-map sqr (list 3))))
= (cons 1 (cons 4 (cons 9 (my-map sqr empty))))
= (cons 1 (cons 4 (cons 9 empty))) = '(1 4 9)
```

Two clauses, one per shape of a list, and `f` is passed down unchanged.
The recursive call is an argument to `cons`, so it is not a tail call.

The Model at Work, Again: A Claim About `my-map`

A test checks one list: `(length (my-map sqr '(1 2 3)))` is `3`.
The model checks every list, if we trace with letters instead of numbers.

The Model at Work, Again: A Claim About `my-map`

A test checks one list: `(length (my-map sqr '(1 2 3)))` is `3`.
The model checks every list, if we trace with letters instead of numbers.

Claim

For every function `f` and every list `lst`:
`(length (my-map f lst)) = (length lst)`.

The Model at Work, Again: A Claim About `my-map`

A test checks one list: `(length (my-map sqr '(1 2 3)))` is `3`.
The model checks every list, if we trace with letters instead of numbers.

Claim

> For every function `f` and every list `lst`:
> `(length (my-map f lst))  =  (length lst)`.

Where does a claim come from? From the test: write down what you saw hold on the lists you tried, with the numbers replaced by names.

The Model at Work, Again: A Claim About `my-map`

A test checks one list: `(length (my-map sqr '(1 2 3)))` is `3`.
The model checks every list, if we trace with letters instead of numbers.

Claim

> For every function `f` and every list `lst`:
> `(length (my-map f lst)) = (length lst)`.

Where does a claim come from? From the test: write down what you saw hold on the lists you tried, with the numbers replaced by names.

A list is `empty` or `(cons z zs)`, so the proof has two cases.
In the second, we **assume** the claim for the shorter list `zs`, and use it once.

The Model at Work, Again: The Proof



Case `empty`.

The Model at Work, Again: The Proof

Case `empty`.

```
(length (my-map f empty)) = (length empty)        the first clause of my-map
```

The Model at Work, Again: The Proof



Case `empty`.

```
(length (my-map f empty)) = (length empty)
```
the first clause of `my-map`

Case `(cons z zs)`. Assumption: `(length (my-map f zs)) = (length zs)`.

The Model at Work, Again: The Proof

Case `empty`.

```
(length (my-map f empty)) = (length empty)
```
the first clause of `my-map`

Case `(cons z zs)`. Assumption: `(length (my-map f zs)) = (length zs)`.

```
(length (my-map f (cons z zs)))
= (length (cons (f z) (my-map f zs)))
```
the **`else`** clause of `my-map`

The Model at Work, Again: The Proof

Case `empty`.

```
(length (my-map f empty)) = (length empty)
```
the first clause of `my-map`

Case `(cons z zs)`. Assumption: `(length (my-map f zs)) = (length zs)`.

```
(length (my-map f (cons z zs)))
= (length (cons (f z) (my-map f zs)))
= (+ 1 (length (my-map f zs)))
```
the **`else`** clause of `my-map`

`length` of a `cons`

The Model at Work, Again: The Proof

Case `empty`.

```
(length (my-map f empty)) = (length empty)
```
the first clause of `my-map`

Case `(cons z zs)`. Assumption: `(length (my-map f zs)) = (length zs)`.

```
(length (my-map f (cons z zs)))
= (length (cons (f z) (my-map f zs)))
= (+ 1 (length (my-map f zs)))
= (+ 1 (length zs))
```
the **`else`** clause of `my-map`

`length` of a `cons`

the assumption, used here and only here

The Model at Work, Again: The Proof

Case `empty`.

```
(length (my-map f empty)) = (length empty)
```
the first clause of `my-map`

Case `(cons z zs)`. Assumption: `(length (my-map f zs)) = (length zs)`.

```
(length (my-map f (cons z zs)))
= (length (cons (f z) (my-map f zs)))
= (+ 1 (length (my-map f zs)))
= (+ 1 (length zs))
= (length (cons z zs))
```
the **`else`** clause of `my-map`

`length` of a `cons`

the assumption, used here and only here

`length` of a `cons`, backwards

The Model at Work, Again: The Proof

Case `empty`.

```
(length (my-map f empty)) = (length empty)
```
the first clause of `my-map`

Case `(cons z zs)`. Assumption: `(length (my-map f zs)) = (length zs)`.

```
(length (my-map f (cons z zs)))
= (length (cons (f z) (my-map f zs)))
= (+ 1 (length (my-map f zs)))
= (+ 1 (length zs))
= (length (cons z zs))
```
the **`else`** clause of `my-map`
`length` of a `cons`
the assumption, used here and only here
`length` of a `cons`, backwards

Both cases hold, so the claim holds for every list.
Nothing about `f` was used: the claim holds for every **pure** `f` too.

Why *Pure*: Where the Model Stops Applying

A function that prints its argument before returning it:

```
(define (show x) (displayln x) x)   ; displayln prints

(my-map show (reverse '(1 2)))      ; prints 2, then 1
(reverse (my-map show '(1 2)))      ; prints 1, then 2
```

Why *Pure*: Where the Model Stops Applying

A function that prints its argument before returning it:

```
(define (show x) (displayln x) x)   ; displayln prints

(my-map show (reverse '(1 2)))      ; prints 2, then 1
(reverse (my-map show '(1 2)))      ; prints 1, then 2
```

Equal in the model, by the proof shape we just saw, and both return `'(2 1)`. The program still tells them apart: the printing.

Why *Pure*: Where the Model Stops Applying

A function that prints its argument before returning it:

```
(define (show x) (displayln x) x)   ; displayln prints

(my-map show (reverse '(1 2)))      ; prints 2, then 1
(reverse (my-map show '(1 2)))      ; prints 1, then 2
```

Equal in the model, by the proof shape we just saw, and both return `'(2 1)`.
The program still tells them apart: the printing.

The model sees the value of `(show x)` and nothing else.
An **effect** is exactly what the model does not see.

Why *Pure*: Where the Model Stops Applying

A function that prints its argument before returning it:

```
(define (show x) (displayln x) x)   ; displayln prints

(my-map show (reverse '(1 2)))      ; prints 2, then 1
(reverse (my-map show '(1 2)))      ; prints 1, then 2
```

Equal in the model, by the proof shape we just saw, and both return `'(2 1)`.
The program still tells them apart: the printing.

The model sees the value of `(show x)` and nothing else.
An **effect** is exactly what the model does not see.

The rule

A proof in the model is a proof about **pure** functions.
Elsewhere, check for effects first.

Quick recap

1. What makes a function **higher-order**?
2. Why can `or` not be passed to `apply`?
3. **Non-empty lists** in Racket are a special instance of what?
4. Which pattern from lecture 2 does `foldl` capture?
5. In a proof for every list, what is **assumed**, and where is it used?
6. Why does “for every `f`” need `f` to be **pure**?

Quick recap

1. What makes a function **higher-order**?
   It accepts a function, or returns one.
2. Why can **`or`** not be passed to `apply`?
3. **Non-empty lists** in Racket are a special instance of what?
4. Which pattern from lecture 2 does `foldl` capture?
5. In a proof for every list, what is **assumed**, and where is it used?
6. Why does “for every `f`” need `f` to be **pure**?

Quick recap

1. What makes a function **higher-order**?
   It accepts a function, or returns one.
2. Why can **`or`** not be passed to `apply`?
   It is a special form, not a function.
3. **Non-empty lists** in Racket are a special instance of what?

4. Which pattern from lecture 2 does `foldl` capture?

5. In a proof for every list, what is **assumed**, and where is it used?

6. Why does “for every `f`” need `f` to be **pure**?

Quick recap

1. What makes a function **higher-order**?
   It accepts a function, or returns one.
2. Why can **`or`** not be passed to `apply`?
   It is a special form, not a function.
3. **Non-empty lists** in Racket are a special instance of what?
   Pairs: a value and a list.
4. Which pattern from lecture 2 does `foldl` capture?

5. In a proof for every list, what is **assumed**, and where is it used?

6. Why does “for every `f`” need `f` to be **pure**?

Quick recap

1. What makes a function **higher-order**?
   It accepts a function, or returns one.
2. Why can **`or`** not be passed to `apply`?
   It is a special form, not a function.
3. **Non-empty lists** in Racket are a special instance of what?
   Pairs: a value and a list.
4. Which pattern from lecture 2 does `foldl` capture?
   The accumulator parameter, with the step passed in as a function.
5. In a proof for every list, what is **assumed**, and where is it used?

6. Why does “for every `f`” need `f` to be **pure**?

Quick recap

1. What makes a function **higher-order**?
   It accepts a function, or returns one.
2. Why can **`or`** not be passed to `apply`?
   It is a special form, not a function.
3. **Non-empty lists** in Racket are a special instance of what?
   Pairs: a value and a list.
4. Which pattern from lecture 2 does `foldl` capture?
   The accumulator parameter, with the step passed in as a function.
5. In a proof for every list, what is **assumed**, and where is it used?
   The claim for the shorter list, used once, in the `cons` case.
6. Why does “for every `f`” need `f` to be **pure**?

Quick recap

1. What makes a function **higher-order**?
   It accepts a function, or returns one.
2. Why can **`or`** not be passed to `apply`?
   It is a special form, not a function.
3. **Non-empty lists** in Racket are a special instance of what?
   Pairs: a value and a list.
4. Which pattern from lecture 2 does `foldl` capture?
   The accumulator parameter, with the step passed in as a function.
5. In a proof for every list, what is **assumed**, and where is it used?
   The claim for the shorter list, used once, in the `cons` case.
6. Why does “for every `f`” need `f` to be **pure**?
   The model replaces a call by its value.
   With an effect, the value is not all the call does.

Quiz for Lecture 3

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 3

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

> Say what you learned and what stayed unclear, **specifically**.
> I read them before writing the next lecture.

What's Next

In today's lab you will

- write the worksheet on this lecture
- defend problem set 2, if your name is drawn
- start on problem set 3

→ **Before next week: iterations and your own datatypes**

- Read SICP §2.2.3 (Abelson and Sussman 1996), sequences as conventional interfaces.
- Read the Racket Guide §5 and §11, at `https://docs.racket-lang.org/guide/`.
- Finish exercises 3.2 to 3.5, and check each on the example from its slide.

Thank you!



References



Abelson, Harold and Gerald Jay Sussman (1996). ***Structure and interpretation of computer programs.*** The MIT Press.